GRAVEATLAS

Phase 4 — GitHub Publication, Data Pipeline & Automated Content Release

Full copy-paste implementation prompt for the Base44 agent.

COPY-PASTE MASTER PROMPT

```
GRAVEATLAS — PHASE 4
GITHUB PUBLICATION, DATA PIPELINE & AUTOMATED CONTENT RELEASE

Phase 1, Phase 2 and Phase 3 must already be complete.

Continue from the existing GraveAtlas implementation.

IMPORTANT:
- Do NOT rebuild Phases 1–3.
- Reuse the existing Android app, Cloudflare Worker/API, GitHub App, public data repository, schemas, search, map, authentic
- Do NOT start Phase 5.
- Do NOT add paid services without explicit approval.
- Do NOT expose GitHub credentials to the Android client.
- Do NOT give ordinary users direct GitHub repository write access.
- Do NOT publish unapproved or unvalidated data.
- Do NOT fabricate cemetery, grave, person or historical information.
- Preserve provenance, audit history and data integrity.
- Do NOT perform irreversible bulk operations without explicit approval.
- Complete only Phase 4, then stop.

================================================
PART 1 — PUBLIC DATA REPOSITORY CONTRACT
================================================

Review and finalize the public GitHub repository contract.

Define:

- repository purpose
- directory structure
- data format
- schema location
- validation rules
- metadata
- documentation
- versioning convention

Keep public data separate from private application data.

Never place:

- private contributor data
- moderator notes
- authentication secrets
- API keys
- GitHub private keys
- ADMIN_TOKEN

in the public repository.

================================================
PART 2 — DATA FILE STRUCTURE
================================================

Define a predictable machine-readable structure.

Organize data by an appropriate stable key such as:

country
region
city
cemetery
or another architecture-supported identifier.

Avoid creating thousands of unnecessary files when a better structure already exists.

Do not reorganize existing production data destructively.
```

```
================================================
PART 3 — SCHEMA VERSIONING
================================================

Define a version for the public data schema.

A record must be distinguishable from:

- schema version
- dataset version
- application version

Document compatibility rules.

================================================
PART 4 — PUBLIC DATA VALIDATION
================================================

Before publication, validate:

- JSON/YAML/CSV syntax where applicable
- schema
- required fields
- identifiers
- dates
- coordinates
- references
- provenance
- duplicate identifiers
- prohibited private fields

Invalid data must block publication.

================================================
PART 5 — PUBLICATION PREPARATION
================================================

Create the controlled publication pipeline:

APPROVED CONTRIBUTION
→ NORMALIZE
→ VALIDATE
→ GENERATE DATA CHANGE
→ VALIDATE AGAIN
→ PREPARE COMMIT/PULL REQUEST
→ REVIEW/APPROVAL AS REQUIRED
→ PUBLISH

Do not skip validation.

================================================
PART 6 — GITHUB APP AUTHENTICATION
================================================

Use the existing GitHub App.

Credentials must remain server-side.

Use:

- GITHUB_APP_ID
- GITHUB_PRIVATE_KEY
- GITHUB_INSTALLATION_ID

through secure secret/environment configuration.

Never put them into:

- Android source
- APK/AAB
- public repository
- logs
- API responses

================================================
PART 7 — INSTALLATION PERMISSIONS
================================================

Verify the GitHub App has only the permissions required.
```

Use least privilege.

Do not request broad repository or organization permissions unnecessarily.

Document actual permissions.

==========
PART 8 — REPOSITORY ACCESS
==========

Verify backend access to the intended repository.

Handle:

- repository not found
- permission denied
- authentication failure
- rate limit
- API failure

with safe errors.

Never fall back to user-supplied GitHub credentials.

==========
PART 9 — CHANGE GENERATION
==========

Generate deterministic changes from approved data.

A contribution should produce a predictable change.

Avoid unnecessary formatting changes that create noisy commits.

Do not rewrite unrelated records.

==========
PART 10 — DUPLICATE PUBLICATION PROTECTION
==========

Prevent the same approved contribution from being published twice.

Use stable identifiers such as:

- contribution ID
- publication ID
- commit reference
- idempotency key

where appropriate.

A retry must not create duplicate public records.

==========
PART 11 — COMMIT / PULL REQUEST STRATEGY
==========

Use the existing repository workflow.

If direct commits are authorized:

ensure only approved, validated changes are committed.

If pull requests are used:

create a controlled PR containing only the approved changes.

Do not merge automatically unless the existing policy explicitly permits it.

==========
PART 12 — COMMIT METADATA
==========

Use meaningful commit messages.

Do not include:

- passwords
- tokens
- private user information

- moderation notes

Commit metadata should identify the operation without exposing sensitive information.

==========
PART 13 — PUBLICATION AUDIT
==========

Record:

- contribution ID
- publication ID
- commit/PR identifier
- timestamp
- result
- error if failed

Keep internal audit information private.

==========
PART 14 — PUBLICATION STATES
==========

Use clear states such as:

APPROVED
QUEUED
PUBLISHING
PUBLISHED
FAILED
RETRYING

Do not mark a contribution PUBLISHED until publication is confirmed.

==========
PART 15 — FAILURE RECOVERY
==========

If GitHub publication fails:

- preserve approved contribution
- preserve audit event
- preserve failure reason
- retry safely where appropriate
- prevent duplicate publication

Do not lose approved data because GitHub is temporarily unavailable.

==========
PART 16 — RETRY POLICY
==========

Implement bounded retries.

Classify errors:

RETRYABLE
NON_RETRYABLE

Use safe backoff.

Do not retry authentication or validation errors forever.

==========
PART 17 — GITHUB RATE LIMITING
==========

Handle GitHub API rate limits.

When rate limited:

- preserve pending publication
- delay retry
- report status
- avoid request storms

==========
PART 18 — DATA MERGE SAFETY
==========

When publishing an update to an existing public record:

verify the target record.

Do not blindly overwrite a newer public version.

Where a conflict exists:

send it to controlled review.

==========
PART 19 — PROVENANCE PRESERVATION
==========

Published records must preserve appropriate provenance.

Do not remove existing source information when applying an update.

If a new source is added, retain the historical provenance where appropriate.

==========
PART 20 — PUBLIC DATA DIFF
==========

Before publication, generate or inspect a change summary.

Show:

- added records
- modified records
- removed records if any
- metadata changes

Unexpected large changes should be blocked or flagged.

==========
PART 21 — MASS CHANGE PROTECTION
==========

Implement a safety threshold for unexpectedly large publication changes.

If a contribution or operation attempts to modify an unusually large number of records:

- stop
- flag
- require authorized review

Do not silently publish mass changes.

==========
PART 22 — PUBLICATION WORKER/JOB
==========

If background publication jobs exist:

ensure jobs have:

- unique ID
- state
- retry count
- timestamps
- error state
- idempotency protection

Do not create uncontrolled infinite jobs.

==========
PART 23 — PUBLICATION QUEUE
==========

If a queue is used:

support:

QUEUED
PROCESSING
SUCCESS
FAILED

Avoid processing the same job concurrently unless the operation is safely idempotent.

==========
PART 24 — DATASET VERSIONING
==========

Track dataset versions.

A dataset version should identify a known public state.

Document:

- version format
- release relationship
- migration rules

Do not confuse dataset version with Android version.

==========
PART 25 — CHANGELOG
==========

Maintain a public changelog where appropriate.

Include:

- dataset release
- important data changes
- schema changes
- application-impacting changes

Do not publish private moderation information.

==========
PART 26 — AUTOMATED VALIDATION
==========

Create GitHub Actions or existing repository automation where appropriate for:

- schema validation
- data validation
- duplicate detection
- formatting
- security scanning
- repository integrity

Use free/existing GitHub functionality where practical.

==========
PART 27 — RELEASE ARTIFACT / DATA SNAPSHOT
==========

If the architecture uses dataset releases or snapshots:

ensure they are reproducible and identifiable.

Do not create unnecessary large artifacts.

==========
PART 28 — PUBLIC API SYNCHRONIZATION
==========

After a successful public dataset change:

ensure API/public search data can recognize the updated dataset according to the existing architecture.

Handle cache invalidation safely.

Do not claim synchronization until it actually occurs.

==========
PART 29 — ANDROID REFRESH
==========

Ensure Android users can eventually see newly published public information.

Handle:

- cached data
- refresh
- stale data
- API errors

Do not require a new APK release for every public data update unless the architecture requires it.

==========
PART 30 — SECURITY REVIEW
==========

Verify:

- GitHub App credentials remain server-side
- publication endpoints require authorization
- ordinary users cannot trigger arbitrary repository writes
- repository paths are validated
- commit content is validated
- path traversal is impossible
- untrusted input cannot alter arbitrary repository files

==========
PART 31 — TESTING
==========

Test:

- valid publication
- invalid publication
- GitHub authentication failure
- repository permission failure
- rate limit
- duplicate publication
- retry
- merge conflict
- mass-change protection
- provenance preservation
- cache refresh
- unauthorized publication attempt

Do not modify real production data destructively during testing.

==========
PART 32 — DOCUMENTATION
==========

Create/update:

docs/PUBLICATION-PIPELINE.md
docs/GITHUB-PUBLISHING.md
docs/DATASET-VERSIONING.md
docs/PUBLIC-DATA-REPOSITORY.md
docs/PUBLICATION-FAILURE-RECOVERY.md
docs/DATA-RELEASES.md

Documentation must match actual implementation.

==========
PHASE 4 ACCEPTANCE GATE
==========

Verify:

[ ] Public repository contract
[ ] Data file structure
[ ] Schema versioning
[ ] Validation
[ ] Publication preparation
[ ] GitHub App authentication
[ ] Least-privilege permissions
[ ] Repository access
[ ] Deterministic changes
[ ] Duplicate publication protection
[ ] Commit/PR strategy
[ ] Commit metadata
[ ] Publication audit
[ ] Publication states
[ ] Failure recovery
[ ] Retry policy
[ ] Rate-limit handling
[ ] Merge safety
[ ] Provenance preservation
[ ] Public diff
[ ] Mass-change protection
[ ] Publication jobs

[ ] Publication queue
[ ] Dataset versioning
[ ] Changelog
[ ] Automated validation
[ ] Dataset release/snapshot
[ ] API synchronization
[ ] Android refresh
[ ] Security review
[ ] Tests
[ ] Documentation

==========
FINAL REPORT
==========

Return:

PHASE 4 STATUS:
READY / NOT READY

DATA PIPELINE:
PASS / FAIL

VALIDATION:
PASS / FAIL

GITHUB APP:
PASS / FAIL

PUBLICATION:
PASS / FAIL

DUPLICATE PROTECTION:
PASS / FAIL

PROVENANCE:
PASS / FAIL

VERSIONING:
PASS / FAIL

QUEUE/JOBS:
PASS / FAIL / NOT IMPLEMENTED

FAILURE RECOVERY:
PASS / FAIL

SECURITY:
PASS / FAIL

API SYNCHRONIZATION:
PASS / FAIL

ANDROID REFRESH:
PASS / FAIL

TESTS:
PASS / FAIL

DOCUMENTATION:
PASS / FAIL

==========
ACTUAL TEST RESULTS
==========

Report actual results only.

Validation:
[actual result]

GitHub authentication:
[actual result]

Publication:
[actual result]

Duplicate protection:
[actual result]

Retry:
[actual result]

Conflict handling:
[actual result]

Mass-change protection:
[actual result]

Provenance:
[actual result]

Cache/API synchronization:
[actual result]

Security:
[actual result]

Do NOT invent results.

==========
BLOCKERS
==========

List every issue preventing:

PHASE 4 = READY

For each:
- severity
- component
- problem
- impact
- recommended remediation

==========
NON-BLOCKING ISSUES
==========

List remaining improvements.

==========
STOP CONDITION
==========

Do NOT start Phase 5.

Do NOT publish unapproved data.

Do NOT perform destructive bulk operations.

Do NOT expose GitHub credentials.

Do NOT add paid services without explicit approval.

Do NOT fabricate data or test results.

Do NOT delete legitimate cemetery or historical data.

STOP after the final report.

WAIT FOR EXPLICIT INSTRUCTIONS FOR PHASE 5.